

JAVA PROGRAMMING LABORATORY

Experiment No.	06
Date	23/09/26
Title	Implementation of PriorityQueue Methods in Java

Objective / Aim

To write and execute a Java program that demonstrates commonly used methods of the PriorityQueue class, including add(), offer(), peek(), contains(), size(), remove(), and poll().

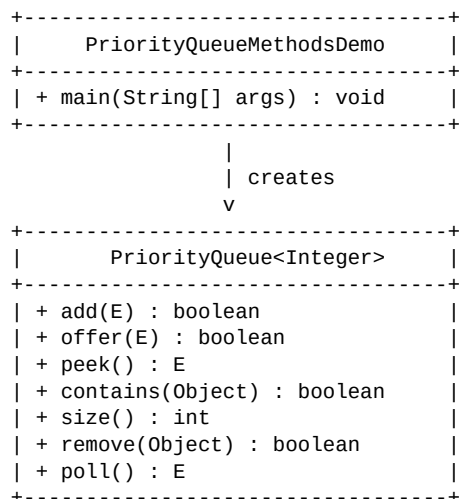
System Requirements

Hardware Requirements	Software Requirements
Computer/Laptop	Operating System: Windows/Linux/macOS
Processor: Intel Core i3 or above	JDK 17 or JDK 21
RAM: 4 GB or above	Java compiler (javac)
Storage: At least 1 GB free space	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code

Algorithm / Logic Description

1. Import PriorityQueue from the java.util package.
2. Create a PriorityQueue named tasks.
3. Insert 40, 10, and 30 using add() and display the queue.
4. Insert 20 using offer() and display the queue.
5. Use peek() to display the head without removing it.
6. Use contains() to check whether 30 and 50 are present.
7. Use size() to display the number of elements.
8. Use remove(30) to remove the specified element.
9. Use poll() to remove and display the head element.
10. Display the remaining queue and final size.

UML Class Diagram



Source Code

```
import java.util.PriorityQueue;

class PriorityQueueMethodsDemo {

    public static void main(String[] args) {

        // Creating a PriorityQueue
        PriorityQueue<Integer> tasks = new PriorityQueue<>();

        // add(E e)
        tasks.add(40);
        tasks.add(10);
        tasks.add(30);

        System.out.println("After add(): " + tasks);

        // offer(E e)
        tasks.offer(20);
        System.out.println("After offer(): " + tasks);

        // peek()
        System.out.println("Head using peek(): " + tasks.peek());
        System.out.println("Queue after peek(): " + tasks);

        // contains(Object o)
        System.out.println("Contains 30? " + tasks.contains(30));
        System.out.println("Contains 50? " + tasks.contains(50));

        // size()
        System.out.println("Size of PriorityQueue: " + tasks.size());

        // remove(Object o)
        tasks.remove(30);
        System.out.println("After remove(30): " + tasks);

        // poll()
        System.out.println("Removed head using poll(): " + tasks.poll());
        System.out.println("Queue after poll(): " + tasks);

        // Checking size again
        System.out.println("Final size: " + tasks.size());
    }
}
```

Completion and Execution Commands

Save as:
PriorityQueueMethodsDemo.java

Compile:
javac PriorityQueueMethodsDemo.java

Execute:
java PriorityQueueMethodsDemo

Output

```
After add(): [10, 40, 30]
After offer(): [10, 20, 30, 40]
Head using peek(): 10
Queue after peek(): [10, 20, 30, 40]
Contains 30? true
Contains 50? false
Size of PriorityQueue: 4
After remove(30): [10, 20, 40]
Removed head using poll(): 10
Queue after poll(): [20, 40]
Final size: 2
```

Note: PriorityQueue guarantees priority ordering for its head element. The queue's printed representation is not required to be completely sorted.